

Name: Kanika Saini

SAP ID: 590034098

Program: Btech CSE B16

HDOC Day-20 — Menu-Driven C Program

1. Algorithm

Step 1: Start.

Step 2: Input two positive integers m and n .

Step 3: Check whether:

- $m > 0$
- $n > 0$
- $m \leq n$

If not, display "**Invalid range**" and stop.

Step 4: Display the menu:

1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit

Step 5: Read the user's choice.

Step 6: Use a `switch` statement:

- **Case 1:** For every integer from m to n , print its first 10 multiples.
- **Case 2:** For every integer from m to n , add numbers 1 through 10 and display the results.
- **Case 3:** For every integer from m to n , subtract numbers 1 through 10 and display the results.
- **Case 4:** For every integer from m to n , divide it by numbers 1 through 10 and display results up to **2 decimal places**.
- **Case 5:** Exit the program.
- **Default:** Display "**Invalid choice! Please enter 1 to 5.**"

Step 7: Repeat the menu using a `do-while` loop until the user selects 5.

Step 8: Stop.

2. Test-Case Table

Test Case	Input	Choice	Expected Output	Actual Output	Result
1	m=2, n=3	1	Multiplication tables of 2 and 3 up to 10	Same as expected	Pass
2	m=2, n=3	2	Addition tables of 2 and 3 using 1–10	Same as expected	Pass
3	m=2, n=3	3	Subtraction tables of 2 and 3 using 1–10	Same as expected	Pass
4	m=2, n=3	4	Division tables of 2 and 3 using 1–10, to 2 decimal places	Same as expected	Pass
5	m=2, n=3	6	Invalid choice message	Invalid choice message displayed	Pass
6	m=2, n=3	5	Program terminates	Program terminated	Pass

3. C Program

```
#include <stdio.h>

int main()
{
    int m, n, choice, i, j;

    printf("Enter two positive integers m and n (m <= n): ");
    scanf("%d %d", &m, &n);

    if (m <= 0 || n <= 0 || m > n)
    {
        printf("Invalid range!\n");
        return 0;
    }

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Multiplication Tables\n");
        printf("2. Addition Tables\n");
        printf("3. Subtraction Tables\n");
        printf("4. Division Tables\n");
```

```

printf("5. Exit\n");
printf("=====\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        for (i = m; i <= n; i++)
        {
            printf("\nMultiplication Table of %d:\n", i);

            for (j = 1; j <= 10; j++)
            {
                printf("%d x %d = %d\n", i, j, i * j);
            }
        }
        break;

    case 2:
        for (i = m; i <= n; i++)
        {
            printf("\nAddition Table of %d:\n", i);

            for (j = 1; j <= 10; j++)
            {
                printf("%d + %d = %d\n", i, j, i + j);
            }
        }
        break;

    case 3:
        for (i = m; i <= n; i++)
        {
            printf("\nSubtraction Table of %d:\n", i);

            for (j = 1; j <= 10; j++)
            {
                printf("%d - %d = %d\n", i, j, i - j);
            }
        }
        break;

    case 4:
        for (i = m; i <= n; i++)
        {
            printf("\nDivision Table of %d:\n", i);

            for (j = 1; j <= 10; j++)
            {
                printf("%d / %d = %.2f\n",
                    i, j, (float)i / j);
            }
        }
        break;

    case 5:

```

```

        printf("\nExiting program...\n");
        break;

    default:
        printf("\nInvalid choice! Please enter 1 to 5.\n");
    }

    } while (choice != 5);

    return 0;
}

```

4. Compilation and Execution

```

(kali@kali)-[~/Desktop]
$ cc day20.c

(kali@kali)-[~/Desktop]
$ ./a.out
Enter two positive integers m and n (m ≤ n): 2 3

===== MENU =====
1. Multiplication Tables
2. Addition Tables
3. Subtraction Tables
4. Division Tables
5. Exit
=====

Enter your choice: 1

Multiplication Table of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

Multiplication Table of 3:
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30

```

```

Enter your choice: 2

Addition Table of 2:
2 + 1 = 3
2 + 2 = 4
2 + 3 = 5
2 + 4 = 6
2 + 5 = 7
2 + 6 = 8
2 + 7 = 9
2 + 8 = 10
2 + 9 = 11
2 + 10 = 12

Addition Table of 3:
3 + 1 = 4
3 + 2 = 5
3 + 3 = 6
3 + 4 = 7
3 + 5 = 8
3 + 6 = 9
3 + 7 = 10
3 + 8 = 11
3 + 9 = 12
3 + 10 = 13

```

```
Enter your choice:
3
Subtraction Table of 2:
2 - 1 = 1
2 - 2 = 0
2 - 3 = -1
2 - 4 = -2
2 - 5 = -3
2 - 6 = -4
2 - 7 = -5
2 - 8 = -6
2 - 9 = -7
2 - 10 = -8

Subtraction Table of 3:
3 - 1 = 2
3 - 2 = 1
3 - 3 = 0
3 - 4 = -1
3 - 5 = -2
3 - 6 = -3
3 - 7 = -4
3 - 8 = -5
3 - 9 = -6
3 - 10 = -7

Division Table of 2:
2 / 1 = 2.00
2 / 2 = 1.00
2 / 3 = 0.67
2 / 4 = 0.50
2 / 5 = 0.40
2 / 6 = 0.33
2 / 7 = 0.29
2 / 8 = 0.25
2 / 9 = 0.22
2 / 10 = 0.20

Division Table of 3:
3 / 1 = 3.00
3 / 2 = 1.50
3 / 3 = 1.00
3 / 4 = 0.75
3 / 5 = 0.60
3 / 6 = 0.50
3 / 7 = 0.43
3 / 8 = 0.38
3 / 9 = 0.33
3 / 10 = 0.30
```

```
Enter your choice: 6

Invalid choice! Please enter 1 to 5.

===== MENU =====
1. Multiplication Tables
2. Addition Tables
3. Subtraction Tables
4. Division Tables
5. Exit
=====

Enter your choice: 5

Exiting program...
```

5. Test-Case Evaluation

For **Test Case 1**, let:

```
m = 2
n = 3
choice = 1
```

The program correctly produces:

Multiplication Table of 2:

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
...
2 x 10 = 20
```

Multiplication Table of 3:

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
...
3 x 10 = 30
```

Therefore, **Test Case 1: PASS.**

For **Test Case 2**, $m=2$, $n=3$, $choice=2$. The program correctly generates $2+1$ through $2+10$ and $3+1$ through $3+10$. **Result: PASS.**

For **Test Case 3**, $m=2$, $n=3$, $choice=3$. The program correctly calculates subtraction using subtrahends 1–10, including negative answers where applicable. **Result: PASS.**

For **Test Case 4**, $m=2$, $n=3$, $choice=4$. For example:

```
2 / 1 = 2.00
2 / 2 = 1.00
2 / 3 = 0.67
...
3 / 2 = 1.50
3 / 3 = 1.00
```

All results are displayed to **two decimal places** using `%.2f`. **Result: PASS.**

For **Test Case 5**, entering choice 6 executes the `default` case and displays:

```
Invalid choice! Please enter 1 to 5.
```

The menu appears again. **Result: PASS.**

For **Test Case 6**, entering choice 5 displays the exit message and terminates the loop. **Result: PASS.**

Conclusion

The program successfully uses **nested for loops, switch-case, and a do-while loop** to perform all four table operations for integers from m to n . It also handles invalid menu choices and continues execution until the user selects **Exit**.